Time-Adaptive SVDD Control Chart

Algorithm Technical Documentation

Time-Adaptive SVDD for Multivariate Process Monitoring

Language: Python
Algorithm: Time-Adaptive SVDD Control Chart
Date: March 2026

1. Overview

This document describes the algorithmic implementation of the Time-Adaptive SVDD Control Chart, a multivariate statistical process monitoring method. The algorithm fits a time-weighted Support Vector Data Description (SVDD) to a representative reference set selected from historical training data, calibrates a dual-threshold control limit via bootstrap resampling, and applies a three-zone online update rule to monitor incoming test observations.

The control limit is fixed before monitoring begins, computed from the reference set via two alternative bootstrap procedures. During online monitoring, each new observation is classified into one of three zones that determine whether the reference set and SVDD model are updated, and whether an alarm is raised.

Module	Role
Data ingestion and normalisation	Load train/test CSVs; apply joint RobustScaler normalisation
Reference set construction	Select representative subset of training data via R-column quantile banding; random subsample to S0
Time-weighting (timefactor)	Compute exponentially decaying observation weights for SVDD fitting
Bootstrap control limit calibration	Derive dual control limits (cl, ul) via two bootstrap procedures on the reference set
Online monitoring loop	Iterate over test observations; classify into three zones; update reference set and SVDD adaptively

Module	Role
Alarm detection and visualisation	Collect alarm counts; plot distance statistic against fixed control limits

2. Data Ingestion and Normalisation

2.1 Input Files

The pipeline requires two CSV files: `real example-testdata.csv` and `real example-traindata.csv`. The first column of each file is dropped via `.iloc[0:n, 1:]`, treating it as a row-index column. The column `Confirmedcases` is separately extracted from the training file as the one-dimensional array `R`, which is used in the reference set selection step.

Variable	Source	Shape	Role
`y_test`	`real example-testdata.csv`	(n_test, p)	Observations to be monitored online
`y_train`	`real example-traindata.csv`	(n_train, p)	Historical reference data for selection and SVDD fitting
`R`	Column Confirmedcases of traindata	(n_train,)	Univariate selection criterion for IQR banding

2.2 Joint Robust Normalisation

Training and test data are concatenated before scaling so that a single transformation is applied consistently to both sets:

```
All = np.r_[y_test, y_train]
robust = preprocessing.RobustScaler()
All = robust.fit_transform(All)
y_test  = All[:y_test.shape[0], :]
y_train = All[y_test.shape[0]:, :]
```

Fitting the scaler on the combined array prevents information leakage: the median and interquartile range used for scaling are estimated from the full dataset rather than from training data alone. `RobustScaler` centres each feature by its median and scales by its interquartile range, providing resistance to outliers.

3. Reference Set Construction

3.1 Quantile Banding on R

Rather than using SVDD distances (as in the RTSVDD pipeline), this algorithm selects reference observations by applying quantile thresholds directly to the univariate column `R`. Observations whose `R` value falls between the 35th and 50th percentiles of the training `R` distribution are retained:

```
q1 = np.quantile(R, 0.35)
q3 = np.quantile(R, 0.50)
q1_ind = np.where(R >= q1)[0]
q3_ind = np.where(R <= q3)[0]
IQR_ind = np.intersect1d(q1_ind, q3_ind)   # indices satisfying both conditions
y_IQR = y_train[IQR_ind, :]
```

The intersection condition $q1 \leq R \leq q3$ selects observations in the central part of the `R` distribution, excluding both the lowest 35% and the highest 50%. This targets a moderate-activity band in the training period.

3.2 Random Subsampling to S0

From the IQR band, `S0 = 100` observations are selected by shuffling the row indices of `y_IQR` and taking the first 100 elements:

```
row_rand_array = np.arange(y_IQR.shape[0])
np.random.shuffle(row_rand_array)
extract = abs(np.sort(-row_rand_array[0:S0]))
y_train = y_train[extract, :]    # y_train is overwritten with the selected rows
y_S0 = y_train[:S0, :]           # reference set: shape (100, p)
```

Note: The expression `abs(np.sort(-row_rand_array[0:S0]))` sorts the first 100 shuffled indices in ascending order. `y_train` is overwritten in place with the 100 selected rows, and `y_S0` is then taken as the first 100 rows of the overwritten `y_train`. After this step `y_train` no longer contains the full training set; it contains only the selected reference rows.

4. Time-Weighting: timefactor Function

The function `timefactor(b, y_S0)` constructs a weight vector of length `y_S0.shape[0]` that assigns decreasing importance to older observations in the reference set. The weight for the k-th observation (ordered from oldest to newest, $k = 0, ..., n-1$) is:

$$w_k = 1 / (b \cdot (n - 1 - k) + 1), \quad b = 0.4$$

With `b = 0.4` and `n = 100`, the oldest observation (k = 0) receives weight $1/(0.4 \times 99 + 1) = 1/40.6 \approx 0.025$, while the newest (k = 99) receives $1/(0+1) = 1$. The weight vector is not normalised before being passed to `svdd.fit()`; normalisation is handled internally by `BaseSVDD` if required.

```
def timefactor(b, y_S0):
    weight_f_S0 = [[1 / (b*(y_S0.shape[0]-1) + 1)]]   # weight for oldest
    for i in range(y_S0.shape[0]-1):
```

```
        weight_t_S0 = [[1 / (b*(y_S0.shape[0]-i-2) + 1)]]
        weight_f_S0 = np.r_[weight_f_S0, weight_t_S0]
    return weight_f_S0   # shape: (n, 1), oldest-to-newest
```

The function is called once before the bootstrap phase to produce `weight_S0` for the initial SVDD fit on `y_S0`. It is called again inside the monitoring loop whenever the reference set grows beyond its initial size (Zone 2 update), to produce a fresh weight vector for the enlarged `y_S0`.

5. Bootstrap Control Limit Calibration

Two control limits are derived from the reference set before monitoring begins: an upper control limit `cl_S0` (alarm threshold) and a lower update limit `ul_S0` (unconditional update threshold). Two independent bootstrap procedures are implemented; the results of the second procedure are adopted as the operative limits.

5.1 Procedure 1: Distance-Space Bootstrap

An initial SVDD is fitted on `y_S0` with time weights. The distances of all reference observations from the SVDD hypersphere centre are computed and stored in `distance_S0`. Bootstrap resampling is then applied directly to this distance vector:

```
svdd = BaseSVDD(C=0.8, gamma=0.07, kernel='rbf', display='off')
svdd.fit(y_S0, weight=weight_S0)
distance_S0 = svdd.get_distance(y_S0).A.flatten()   # shape: (100,)

for i in range(10000):
    bootstrap_S01 = np.random.choice(distance_S0, len(distance_S0), replace=True)
    cl_S01.append(np.quantile(bootstrap_S01, 1-a))        # 95th percentile
    ul_S01.append(np.quantile(bootstrap_S01, 1-a-d))      # 90th percentile
cl_S01 = np.mean(cl_S01)   # mean over 10,000 bootstrap replicates
ul_S01 = np.mean(ul_S01)
```

Each replicate resamples 100 distances with replacement and computes the $(1-\alpha)$ = 95th and $(1-\alpha-d)$ = 90th percentiles. The means of these percentiles over 10,000 replicates yield `cl_S01` and `ul_S01`. This procedure resamples in distance space, holding the SVDD geometry fixed.

5.2 Procedure 2: Data-Space Bootstrap

The second procedure resamples in the original data space: each replicate draws a new reference set by sampling rows of y_S0 with replacement, refits the SVDD on this bootstrap sample, and computes distances under the new SVDD:

```
for i in range(1000):
    bootstrap_S02 = y_S0[np.random.choice(y_S0.shape[0], y_S0.shape[0]), :]
    svdd.C = 0.5
    svdd.fit(bootstrap_S02, weight=weight_S0)
    distance2 = svdd.get_distance(bootstrap_S02).A.flatten()
    cl_S02.append(np.quantile(distance2, 1-a))
```

```
    ul_S02.append(np.quantile(distance2, 1-a-d))
cl_S02 = np.mean(cl_S02)   # mean over 1,000 bootstrap replicates
ul_S02 = np.mean(ul_S02)
```

The penalty parameter is changed to `C = 0.5` for the bootstrap SVDD fits, differing from the `C = 0.8` used in the initial SVDD. Over 1,000 replicates, both the SVDD geometry and the distance distribution vary, capturing a broader source of uncertainty than Procedure 1.

5.3 Operative Limits

The results of Procedure 2 are adopted as the operative control limits for online monitoring:

```
cl_S0 = cl_S02   # upper control limit: alarm if distance > cl_S0
ul_S0 = ul_S02   # lower update limit: unconditional update if distance <= ul_S0
```

Limit	Variable	Formula	Interpretation
Upper control limit	`cl_S0`	E_B[ˆq_{1-α}]	Alarm threshold; observations above this limit trigger an alarm
Lower update limit	`ul_S0`	E_B[ˆq_{1-α-d}]	Unconditional update threshold; observations below this are absorbed into the reference set without refitting

Both limits are scalar constants fixed before the monitoring loop begins. Unlike the RTSVDD pipeline, the control limit does not adapt during online monitoring; only the reference set and SVDD model may change.

6. Online Monitoring Loop

6.1 Loop Structure

The monitoring loop iterates over each row of the test matrix `X = y_test`. At each step `i`, the single new observation `New_X = X[i:i+1, :]` is scored by computing its mean SVDD distance under the current model:

```
for i in range(X.shape[0]):
    New_X = X[i:i+1, :]
    distance_New = np.mean(svdd.get_distance(New_X))
```

Because `New_X` is a single observation, `svdd.get_distance(New_X)` returns a scalar-valued array and `np.mean` is a no-op identity; the result `distance_New` is a scalar distance value for the current observation.

6.2 Three-Zone Classification and Update Rule

Each observation is classified into one of three zones based on its distance relative to the two fixed limits. The zone determines how the reference set and SVDD are updated:

Zone	Condition	Action on S0	Action on SVDD
Zone 1 (Alarm)	$d > cl_\{S0\}$	S0 unchanged (frozen)	SVDD unchanged; alarm counter incremented
Zone 2 (Marginal)	$ul_\{S0\} < d \leq cl_\{S0\}$	New_X appended to S0 (S0 grows by 1)	SVDD refit on enlarged S0 with fresh time weights
Zone 3 (In-control)	$d \leq ul_\{S0\}$	New_X appended to S0 (S0 grows by 1)	SVDD unchanged

```
if distance_New > cl_S0:
    distance_f.append(distance_New)
    n_alarm += 1
    # S0 and SVDD unchanged

elif distance_New <= ul_S0:
    distance_f.append(distance_New)
    y_S0 = np.r_[y_S0, New_X]          # Zone 3: append, no refit

elif distance_New > ul_S0 and distance_New <= cl_S0:
    distance_f.append(distance_New)
    y_S0 = np.r_[y_S0, New_X]          # Zone 2: append
    weight_S0 = timefactor(b, y_S0)    # recompute weights for enlarged S0
    svdd.C = 0.5
    svdd.fit(y_S0, weight=weight_S0)   # refit SVDD
```

6.3 Reference Set Growth

Unlike the RTSVDD pipeline, this algorithm does not impose a fixed reference set size: `y_S0` grows by one row whenever a Zone 2 or Zone 3 observation is encountered, and is never truncated. As the reference set expands, the time-weight vector is recomputed (Zone 2 only) to maintain the recency-weighting property for the enlarged set. In Zone 3, the SVDD is not refit; the existing model continues to be used for scoring until the next Zone 2 event triggers a refit.

7. Output Collection

After the loop, the collected per-step distances and alarm indices are assembled:

```
distance_f = np.array([distance_f]).T           # shape: (n_test, 1)
n_alarm_indicate = np.where(distance_f > cl_S0)[0]   # alarm step indices

n = distance_f.shape[0]
radius_cl = np.ones((n, 1)) * cl_S0    # constant upper limit, broadcast
```

```
radius_ul = np.ones((n, 1)) * ul_S0     # constant lower limit, broadcast
```

Output Variable	Shape	Content
`distance_f`	(n_test, 1)	SVDD distance statistic at each test step
`n_alarm`	scalar	Total count of Zone 1 alarm events
`n_alarm_indicate`	(n_alarm,)	Step indices where distance_f > cl_S0
`radius_cl`	(n_test, 1)	Upper control limit cl_S0 broadcast to full length
`radius_ul`	(n_test, 1)	Lower update limit ul_S0 broadcast to full length (computed but not plotted)

8. Visualisation

The function `plot_distance(radius, distance)` is called with `radius_cl` and `distance_f`. It renders the SVDD distance trajectory against the upper control limit as a time-series chart. The lower limit `radius_ul` is computed but not included in the plot.

Element	Style	Represents
Upper control limit	Red solid line, linewidth 3	`radius_cl`
Distance statistic	Black dotted line with circular markers	`distance_f`

The x-axis tick labels map integer indices `[0, 49, 99, 149, 199, 249, 299]` to the date strings `['1-Mar', '20-Apr', '9-Jun', '29-Jul', '17-Sept', '6-Nov', '26-Dec']`. These labels are hardcoded to the reference dataset and should be adjusted for other time series.

9. Hyperparameter Summary

Parameter	Value	Where Used	Role
S0	100	Reference set size	Initial number of observations in y_S0; grows during monitoring
q1 percentile	0.35	IQR banding on R	Lower quantile threshold for selecting training observations
q3 percentile	0.50	IQR banding on R	Upper quantile threshold for selecting training observations
b (time decay)	0.4	timefactor function	Decay rate for reference-set time weights; larger b gives

Parameter	Value	Where Used	Role
			stronger recency bias
C (initial SVDD)	0.8	First SVDD on y_S0	Penalty parameter for initial reference-set fit; tighter boundary
gamma (initial SVDD)	0.07	First SVDD on y_S0	RBF kernel width for initial fit
C (bootstrap / monitoring)	0.5	Bootstrap Procedure 2 and Zone 2 refits	Penalty parameter for bootstrap and online refits
B1 (bootstrap replicates)	10,000	Procedure 1 bootstrap	Number of distance-space bootstrap replicates
B2 (bootstrap replicates)	1,000	Procedure 2 bootstrap	Number of data-space bootstrap replicates
a (significance)	0.05	Both bootstrap procedures	Upper tail probability; control limit at (1-a) quantile
d (gap)	0.05	Both bootstrap procedures	Gap between control limit and update limit; ul at (1-a-d) quantile

10. Dependencies and Execution

10.1 Required Packages

Package	Usage
numpy	Array operations, random sampling, quantile computation
pandas	CSV ingestion
src.BaseSVDD	SVDD fitting and distance computation (custom/local module)
sklearn.preprocessing.RobustScaler	Joint normalisation of train and test data
matplotlib / pylab	Control chart visualisation; custom x-axis tick labels

10.2 Execution

Place `real example-traindata.csv` and `real example-testdata.csv` in the working directory, ensure `src/BaseSVDD.py` is accessible via `sys.path`, and execute:

```
python Time-adaptive SVDD.py
```

References

[1] Tax, D. M. J., & Duin, R. P. W. (2004). Support vector data description. *Machine Learning*, 54(1), 45–66.

[2] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.